ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

Organisation of Islamic Cooperation (OIC)

Department of Computer Science and Engineering (CSE)

Project Report

DORMATRIX

IUT Female Dormitory Management System

SWE 4304: Software Project Lab I

Team Members

Student ID	Team Member
230042114	Procheta Silvie
230042135	Khadiza Sultana
230042139	Sayma Tasnim
230042141	Ayman Binta Altaf Nondiny

Supervisor

Md. Tariquzzaman

Junior Lecturer
Department of Computer Science and Engineering
Islamic University of Technology

Project Title	DORMATRIX: IUT Female Dormitory Management System
Course	SWE 4304 - Software Project Lab I
Source Repository	`github.com/tayma-06/Dormatrix`
Submission Date	March 30, 2026

Contents

Executive Summary 1

1 Introduction 1

1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Motivation . . . 1
1.4 Potential Solution . . . 2
1.5 Objectives . . . 2
1.6 Implemented Features . . . 3
1.7 Application Domain . . . 3

2 Project Overview 3

2.1 Primary User Roles . . . 3
2.2 High-Level Workflow . . . 4

3 Requirements Analysis 4

3.1 Functional Requirements . . . 4
3.2 Non-Functional Requirements . . . 6
3.3 Assumptions and Constraints . . . 6

4 Tools and Technology 7

5 System Architecture and Design 7

5.1 Architectural Pattern . . . 7
5.2 Codebase and Package Distribution . . . 8
5.3 Custom-Built Libraries . . . 9
5.4 Data Storage Design . . . 10
5.5 Security and Credential Handling . . . 11
5.6 Key Algorithms and Design Logic . . . 11
5.6.1 HashFunction: Three-Channel Password Hashing . . . 12
5.6.2 First-Fit Slot Allocation . . . 13
5.6.3 SlotAllocator: Stopwatch Cycle Clock . . . 13
5.6.4 Complaint Policy . . . 14

5.6.5 TimeManager: Clock 15
5.6.6 LaundryController: Timer-Based Auto-Release 15

6 Complaint and Scheduling Integration: A Closed-Loop Operational Design 15
6.1 The Complaint Lifecycle as a Scheduling Driver 15
6.2 Auto-Planning: Time-Aware Forward Search 16
6.3 Bi-Directional Routine Injection 16
6.4 Scheduling Preference and Reschedule Badges 16

7 Utility Infrastructure: The `utils` Package 16
7.1 ConsoleUtil — ANSI Terminal Primitives 16
7.2 TerminalUI and TerminalUIExtras — Bordered Panels and Themed Components 16
7.3 BackgroundFiller — Role-Aware Background Rendering 17
7.4 InputHelper and FastInput — Keyboard and Password Handling 17
7.5 RoleMapper — Role Name Normalization 17
7.6 CafeteriaAsciiUI — Animated Progress Bar 17
7.7 TimeManager as a Shared Service 17

8 How Dormatrix Behaves Like a Real-World System 17
8.1 Time-Gated Cafeteria Purchases 17
8.2 Worker-Student Schedule Conflict Avoidance 17
8.3 Laundry Machine Auto-Release 17
8.4 Emergency Complaint Escalation 18
8.5 Ramadan Schedule Switching 18
8.6 Store Credit and Due Tracking 18
8.7 Fridge Allocation Without 18
8.8 Student Routine as a Living Document 18

9 Module-Wise Implementation Details 18
9.1 Authentication Module 18
9.2 Dashboard and Terminal UI Module 19
9.3 Room Management Module 19
9.4 Facility Booking Module 20
9.5 Store, Balance, and Due Management Module 23

9.6 Cafeteria and Meal Token Module 24
9.7 Complaint Management Module 26
9.8 Lost and Found Module 27
9.9 Announcement Module 29
9.10 Routine, Contacts, and Profile Management 29
9.11 Administrative Control Module 29

10 User Flow and Interface **29**
10.1 Entry and Login 30
10.2 Student 30
10.3 Attendant 32
10.4 Maintenance Worker 33
10.5 Store-in-Charge 33
10.6 Cafeteria Manager 34
10.7 Hall Office 34
10.8 Admin 35
10.9 Interface Design 35

11 Implementation Evidence and Technical Notes **36**
11.1 Entry Point and Runtime Setup 36
11.2 Run Environment 36
11.3 Error Handling 36
11.4 Persistence Transparency 36

12 Testing and Quality Assurance **36**
12.1 Testing Scope 37
12.2 What the Test Suites Cover 37
12.3 Testing Interpretation 37
12.4 Manual Validation 37
12.5 Known Risk Areas 38

13 Contribution **38**
13.1 Team Contribution Overview 38
13.2 Project Timeline 40

14 Future Work **40**

14.1 Current Limitations 41

14.2 Future Work 41

15 Personal Reflections **42**

15.1 Procheta Silvie (230042114) 42

15.2 Khadiza Sultana (230042135) 42

15.3 Sayma Tasnim (230042139) 43

15.4 Ayman Binta Altaf Nondiny (230042141) 44

16 Conclusion **45**

Executive Summary

Dormatrix is a command-line dormitory management system for the IUT Female Dormitory. It brings the main services of the hall into one place: login, room information, room-change handling, complaints handling, facility booking, store and dues, meal tokens, announcements, routines, emergency contacts, and lost-and-found records. Different roles get different dashboards, so each user sees only the work that is related to them.

The project is file-based and uses an MVC (Models, Views, Controllers) and Repository structure. We also built several internal classes ourselves, including `MyArrayList`, `MyString`, `MyOptional`, `HashFunction`, `FirstFitAllocator`, `LastFitAllocator`, and `SlotAllocator`. The only outside library is **JLine3**, and that is used only for masked password input. The cafeteria module uses a time aware clock while the study room uses a stopwatch based clocking system, and complaint scheduling checks a weekly student routine before fixing a visit.

1 Introduction

1.1 Background

Hall life depends on many small services working properly: room management, complaints, food, store purchases, shared facilities, announcements, and emergency communication. In real life, a lot of these things are still handled manually. That usually leads to delay, confusion, and poor follow-up.

Dormatrix was built to make those everyday hall tasks easier to manage in one system. A student can check room information, book a facility, submit a complaint, buy meal tokens, and see announcements from the same application. Staff members get their own role-based dashboards, so they can do their work without extra clutter.

1.2 Problem Statement

The main problems we focused on in this project are:

- Complaints are difficult to submit, assign, and follow up
- Shared facilities can clash without proper booking rules
- Store purchases, balances, and dues are hard to keep organized
- Meal menu and token handling are inconvenient when one manually
- Important hall updates are easy to miss

1.3 Motivation

The idea for this project came from real hall problems we have already faced as IUT Female Hall residents. We wanted to build something that actually fits our dorm life. Our goal

was to make daily hall tasks easier, clearer, and more organized.

1.4 Potential Solution

The solutions we came up for the mentioned problems are mentioned below:

Table 1: Implemented problems and how Dormatrix addresses them

Problem	How Dormatrix addresses it
Complaint handling	Students can submit complaints, attendants can assign workers, workers can update status, and visits can be scheduled and rescheduled.
Room administration	Students can view room information, while the Hall Office can add rooms, assign rooms to unassigned students, and review room-change requests.
Shared facility use	Laundry, study room, and fridge usage are handled through booking and slot logic so clashes are reduced.
Store record keeping	Inventory, purchases, balances, and dues are stored in one system.
Food and token management	The cafeteria module shows menus, supports meal token purchase, and lets the cafeteria side verify tokens.
Hall announcement	Announcements and related dashboard views make important updates easier to see from inside the system.

1.5 Objectives

The main objectives of Dormatrix are:

- Designing a single unified dormitory management platform that serves all hall stakeholders through role-based access
- Applying object-oriented design principles and modular package organization in a real, working project
- Building a file-persistent system that can run without any database setup or external server dependency
- Ensuring that each user role receives a tailored dashboard, appropriate permissions, and a focused set of actions
- Improving visibility, accountability, and coordination across complaints, facility usage, store operations, and cafeteria services

- Producing a polished, demonstration-ready system with maintainable code and comprehensive supporting documentation

1.6 Implemented Features

Dormatrix is intentionally focused on the female dormitory context at IUT. The scope of the current implementation includes:

- User authentication and role-based access control
- Room status viewing, room change applications, and room administration
- Complaint submission, emergency prioritization, worker assignment, and life-cycle tracking
- Facility booking for laundry machines, study room seats, and fridge slots
- Cafeteria token purchase, weekly menu viewing, token verification, and Ramadan schedule switching
- Departmental store inventory, shopping cart, purchase processing, student balances, and due tracking
- Lost-and-found item reporting and claiming
- Hall-wide announcements, maintenance schedules, student routines, emergency contacts, and profile editing

1.7 Application Domain

Dormatrix operates within the **University Dormitory Management** and **Student Residence Administration** domains. This sector focuses on digitizing and coordinating the daily logistics of university housing.

2 Project Overview

2.1 Primary User Roles

The system supports seven operational roles. Each one has its own dashboard layout, and a set of tasks that matches what that role actually needs to do.

Table 2: Role summary within Dormatrix

Role	Options	Primary responsibilities
Student	11	Room status, facility booking, token purchase, store, dues, complaints, routine, announcements, contacts, profile
Attendant	7	Complaint monitoring, worker scheduling, found-item logging, announcement posting, emergency contacts
Maintenance Worker	4	Work field awareness, assigned task handling, visit schedule tracking
Store-in-Charge	4	Inventory control, purchase order processing, sales reporting
Hall Office	6	Room availability checking, room assignment, and room-change review
Admin	4	Account creation and deletion, account view and search
Cafeteria Manager	5	Weekly menu updates, special events, token verification, Ramadan mode toggle

2.2 High-Level Workflow

At a high level, Dormatrix works in a simple way:

1. The user selects a role.
2. The system checks the login information.
3. The correct dashboard opens for that role.
4. The user performs tasks allowed for that role.
5. The controllers update the data and save the changes to text files.
6. Other roles can then see the updated information where needed.

3 Requirements Analysis

3.1 Functional Requirements

The functional requirements were realized as distinct but interconnected modules. Table 3 summarizes the main functional requirement categories, the actors involved, and the expected system behavior for each.

Table 3: Functional requirement matrix

Requirement area	Main actors	Required system behavior
Authentication and access control	All users	Validate login credentials, apply role-specific routing, support masked password entry, and maintain separate user record files per role
Room information and room change	Student, Hall Office	Show current room status, let students apply for room changes, let the hall office add rooms, check available rooms, assign rooms to unassigned students, and review room-change requests
Facility booking	Student, Attendant	Reserve shared resources, prevent double-booking, and auto-release or persist facility state through file-backed slot records
Complaint management	Student, Attendant, Maintenance Worker	Accept complaint submissions, detect emergency keywords, assign priority and worker field, and track status transitions across the full complaint lifecycle
Store and balance control	Student, Store-in-Charge	Display available inventory, process student purchase orders, maintain store balance ledgers and outstanding dues records, and generate sales history and summaries
Cafeteria and meal token system	Student, Cafeteria Manager	Publish and update weekly menus, enforce time-locked meal windows, issue and track meal tokens, verify token status, and support Ramadan schedule switching
Announcements and schedules	Attendant, Maintenance Worker, Student	Publish and display hall-wide notices, record and display maintenance worker visit schedules, and make operational information visible to relevant roles
Lost & found	Student, Attendant	Allow students to report lost items, enable attendants to register found items, and support claiming with traceable record history

Requirement area	Main actors	Required system behavior
Emergency contacts	Student, Attendant	Display, add, and modify emergency contact lists
Profile management	All roles	Allow users to manage personal contact entries, and support profile detail updates and password changes
Administrative control	Admin	Create and delete user accounts across all roles, view or search account records, and keep user-account management centralized

3.2 Non-Functional Requirements

Beyond feature completion, the project demonstrates a number of important non-functional properties:

- **Usability:** Role-aware dashboards reduce menu clutter significantly. A student sees student options only; a maintenance worker sees task and schedule options only. Arrow-key and numeric navigation further reduces friction during use.
- **Readability:** Visual hierarchy through ANSI color themes, bordered panels, and highlight bars on the active menu item helps users orient themselves quickly within each screen.
- **Portability:** The application requires no database server. All state is persisted in structured text files, allowing the project to run on any machine with Java 17 or higher installed.
- **Maintainability:** Source code is separated into distinct packages for CLI views, controllers, models, repositories, libraries, utilities, and exceptions. This separation makes it straightforward to locate, modify, or extend any individual module.
- **Security:** Passwords are hashed using a custom triple-channel DJB2 + XOR function before being written to disk. Plain-text credentials are never stored anywhere in the system.
- **Demonstrability:** A built-in demo mode compresses meal-time windows so that all food-system state transitions can be observed within a short presentation window.

3.3 Assumptions and Constraints

The project operates under the following technical and domain constraints:

- The system relies on local file-based storage, which simplifies setup but restricts the application to sequential access without concurrent multi-user write support

- The user interface assumes execution within a terminal environment that supports ANSI true-color rendering for optimal visual layout
- The application is designed as a standalone, local executable meant for academic evaluation and demonstration, rather than a production-ready environment
- Core workflows and business logic are heavily customized for the IUT Female Dormitory context and would require refactoring to suit different residential facilities

4 Tools and Technology

Dormatrix was developed primarily using **Java** with a file-based persistence approach, a command-line interface. The main technologies used in the current implementation are mentioned below.

Table 4: External libraries used in Dormatrix

Library	Version / Type	Purpose in the project
JLine3	External JAR	Used for masked password input during login so passwords do not appear on the terminal.
JUnit	Testing library	Used for unit testing different parts of the system during development and verification.

Besides these, the project also relies on standard Java APIs. Some of the main built-in libraries used in the project are:

- `java.io` for file reading and writing;
- `java.time` for date and time handling in cafeteria timing, scheduling, and slot logic;
- `java.util` for scanner input, timers, and general utility support;
- `java.util.Scanner` for terminal input handling in different menus and forms;
- `java.util.Timer` and `TimerTask` for time-based actions such as laundry slot auto-release.

This kept the project lightweight while still covering all required functionality.

5 System Architecture and Design

5.1 Architectural Pattern

Dormatrix follows an MVC + Repository structure. The CLI screens work as the view layer. Controllers handle the logic. Repositories read from and write to the text files. Models hold the data objects used across the system.

This separation made the code easier to organize and extend. When we added a feature, we usually knew where its view, logic, and storage parts should go.

Table 5: Layered architecture overview

CLI Layer (Views) Dashboards, forms, panels, interactive menu flows, and role-specific screen layouts
⇓
Controller Layer (Business Logic) Authentication, room management, facility booking, complaint routing, store processing, cafeteria and food, routine handling, announcements, scheduling, and profile management
⇓
Repository / File Access Layer Structured reading, parsing, and writing of pipe-delimited plain-text records
⇓
Model Layer (Domain Entities) Users, rooms, complaints, meal tokens, weekly menus, balances, dues, store items, routines, visit records, announcements, found items, and emergency contacts

5.2 Codebase and Package Distribution

The source code is split into a few main packages so related files stay together. This made the project easier to manage as it grew.

Table 6: Source package distribution

Package	Role in the system
`cli`	All user-facing dashboards, forms, and screen-specific interaction classes, organized by feature area
`controllers`	Business logic for every major operation: authentication, rooms, food, store, complaints, scheduling, balance, and more
`models`	Domain entities and enumerations representing dormitory state (users, rooms, complaints, items, tokens, etc.)
`repo`	File-backed persistence classes that read and write structured records for each domain
`libraries`	Reusable custom utilities including data structures, hashing, file helpers, logging, and slot allocation
`exceptions`	Domain-specific exception classes for input validation and operational error conditions
`utils`	Shared rendering helpers for ANSI color, terminal sizing, input handling, time management, and role mapping
`module`	An additional service layer for the complaint module, encapsulating policy logic separately from the controller
`tests`	JUnit-based test suites covering core logic, custom libraries, UI utilities, and persistence behavior

5.3 Custom-Built Libraries

One thing that matters in this project is that we did not depend only on ready-made utilities. We built a small internal library set and used those classes across the system. Some of them replace common data helpers, some handle files, and some support project-specific logic like hashing and slot allocation.

Table 7: Custom-built libraries used in Dormatrix

Package	Class	What it does in the project
`libraries/collections`	`MyArrayList`	Our own dynamic list class used in place of `ArrayList` in many parts of the project.
`libraries/collections`	`MyString`	A custom string helper used for parsing, matching, editing text, and converting integers to hex.
`libraries/collections`	`MyOptional`	A simple optional wrapper that helps reduce direct null handling.
`libraries/file`	`FilePaths`	Keeps important file locations in one place.
`libraries/file`	`TextFile`	Handles common text-file reading and writing so the rest of the code stays cleaner.
`libraries/hashing`	`HashFunction`	Our custom password-hashing class used during login. Only the hash is stored, not the plain password.
`libraries/logs`	`Logger`	A small logger used to keep track of important actions such as bookings and auto-release events.
`libraries/slots`	`SlotAllocator`	Base class for slot handling. It also contains the cycle-based time logic used for study room booking.
`libraries/slots`	`FirstFitAllocator`	Finds the first free slot and is mainly used for fridge allocation.

5.4 Data Storage Design

The persistence layer uses structured plain-text files with pipe-delimited (`|`) fields, grouped under the `data/` directory by domain. Each repository class is responsible for reading and writing exactly one file type, so adding a field to a record means modifying the model and its corresponding repository class only.

Table 8: Persistent storage files and their contents

File path	Stored information
`data/users/students.txt`	Student identity, department, hashed password, phone, email, and current room assignment
`data/users/admin.txt`	Administrator account credentials
`data/users/hall_attendants.txt`	Attendant records
`data/users/maintenance_workers.txt`	Worker records with field specialization (e.g., Electrician, Plumber)
`data/users/store_in_charges.txt`	Store staff records
`data/users/hall_officers.txt`	Hall office staff records
`data/users/cafeteria_managers.txt`	Cafeteria manager records
`data/rooms/rooms.txt`	Room identifiers, capacities, and current occupancy counts

File path	Stored information
`data/rooms/room_change_applications.txt`	Pending and processed room-change applications with status
`data/complaints/complaints.txt`	Full complaint stream including category, priority, status, tags (scheduling preferences), and assignment
`data/facility/laundrySlots.txt`	Active laundry-slot occupancy records (index, studentId pairs)
`data/facility/studyRoomSlots.txt`	Study-room seat booking state per slot and seat index
`data/foods/config.txt`	Ramadan mode toggle, persisted across application restarts
`data/foods/tokens.txt`	Meal token records and their current status (Active / Used / Expired)
`data/foods/weekly_menu.txt`	Weekly cafeteria menu entries per day, meal type, and Ramadan flag
`data/inventories/inventory.txt`	Store product catalog with quantities and prices
`data/inventories/balances.txt`	Student stored-value balance records
`data/store/dues.txt`	Outstanding student due ledger
`data/store/sales.txt`	Itemized sales history
`data/routines/student_routines.txt`	Weekly routine entries per student (including injected complaint visit markers)
`data/schedules/worker_visits.txt`	Maintenance worker visit scheduling records (complaint-linked)
`data/announcements/announcements.txt`	Hall notice records with timestamps
`data/contacts/emergency_contacts.txt`	Official and personal emergency contact entries
`data/lostItems.txt`	Student-submitted lost-item reports with descriptions and timestamps
`data/foundItems.txt`	Attendant-logged found-item records with claim status

5.5 Security and Credential Handling

The authentication flow works as follows. When a user submits their password, the `HashFunction` class applies a three-channel hashing procedure described in detail in Section 5.6.1. The result is a positive hexadecimal string that is compared against the stored hash in the role-specific user file. The original password is never written to disk at any point and is discarded from memory immediately after comparison.

5.6 Key Algorithms and Design Logic

5.6.1 HashFunction: Three-Channel Password Hashing

The `HashFunction` class in `libraries/hashing` implements a custom password hashing scheme that combines three independent mixing channels into a single digest. Plain-text passwords are converted to a `MyString` instance (the project's own character-array string type) before being passed to the private `xorHash()` method. The implementation is shown below:

Listing 1: HashFunction.java — three-channel hashing

```java
int hash1 = 5381;        // DJB2 seed (chosen for good avalanche properties)
int hash2 = 0;
int hash3 = length;      // seed with string length for length-sensitivity

for (int i = 0; i < length; i++) {
    char c = chars[i];
    hash1 = ((hash1 << 5) + hash1) ^ c;  // DJB2: hash1*33 XOR c
    hash2 = (hash2 ^ c) * 31;             // XOR-then-multiply mixing
    hash3 = hash3 ^ (c << (i % 16));      // position-dependent bit shift
}

int combinedHash = hash1 ^ hash2 ^ hash3;
int positiveHash = combinedHash & 0x7FFFFFFF;
return MyString.intToHex(positiveHash);
```

The three channels each serve a distinct mixing purpose:

Channel 1 (DJB2 polynomial): `hash1` is initialized to 5381, a prime chosen for its good bit-distribution properties in the original DJB2 algorithm. At each step, the update is $\text{hash1} \leftarrow (\text{hash1} \times 33) \oplus c$, implemented as a left-shift by 5 bits plus the original value, which is arithmetically equivalent to multiplication by 33. The XOR with the current character c ensures that every character in the password contributes to every bit position through cascading polynomial expansion.

Channel 2 (XOR-then-multiply): `hash2` begins at zero and applies $\text{hash2} \leftarrow (\text{hash2} \oplus c) \times 31$ at every step. The XOR before multiplication prevents simple linear relationships between adjacent character values from producing predictable hash differences. Multiplying by 31 (another Mersenne-adjacent prime) further mixes the bits across the integer range after the XOR operation.

Channel 3 (position-sensitive XOR): `hash3` is seeded with the string length, making hashes of different-length strings structurally distinct from the outset. Each character is left-shifted by $(i \bmod 16)$ bits before being XORed into `hash3`. This means that the same character appearing at different positions in the password contributes different bit patterns, making the hash sensitive to character order independently of the other two channels.

Combination and normalization: The three channels are combined by XORing all three

values. The sign bit is cleared using the mask `0x7FFFFFFF` to guarantee a non-negative integer, and the result is converted to an 8-character fixed-width hexadecimal string using `MyString.intToHex()`, which implements the hex digit lookup using a custom `char[]` table without delegating to `Integer.toHexString()`.

The output is always exactly 8 hex characters, making it easy to identify and compare in the text files. No salt is currently applied, which is an acknowledged limitation noted in the future improvements section.

5.6.2 First-Fit Slot Allocation

The `FirstFitAllocator` solves the problem of assigning a shared resource slot to a requesting user without manual intervention. It reads the current slot availability array and performs a linear scan, returning the index of the first position that is `null` (unoccupied). That slot is then immediately marked as occupied and persisted to the relevant facility file.

Listing 2: FirstFitAllocator.java

```java
public int findSlot(String[] slots) throws SlotUnavailableException {
    for (int i = 0; i < slots.length; i++) {
        if (slots[i] == null) {
            return i;
        }
    }
    throw new SlotUnavailableException("No available slots found in fridge.");
}
```

For fridge allocation, students are assigned the lowest-numbered available compartment, ensuring fair sequential distribution. The abstract `SlotAllocator` base class defines the allocation contract, making it easy to introduce alternative strategies such as best-fit or round-robin in the future.

5.6.3 SlotAllocator: Stopwatch Cycle Clock

Beyond first-fit allocation, the `SlotAllocator` abstract base class implements a second, independent time mechanism: a *stopwatch-style cycle clock* that drives study room booking slots in real time.

Listing 3: SlotAllocator.java — cycle clock

```java
public static final int REAL_MINS_PER_DAY   = 12;  // full cycle = 12 real minutes
public static final int REAL_MINS_PER_SLOT  = 2;   // each slot = 2 real minutes
public static final int TOTAL_SLOTS         = 6;   // 6 two-minute slots per cycle

public static int getCurrentSlotIndex() {
    long minuteInHour  = LocalTime.now().getMinute();
    long secondInHour  = LocalTime.now().getSecond();
```

```

    long totalSecondsInHour = (minuteInHour * 60) + secondInHour;
    long secondsInCycle     = totalSecondsInHour % (REAL_MINS_PER_DAY *
        60);

    return (int) (secondsInCycle / (REAL_MINS_PER_SLOT * 60));
}
```

The mechanism works as follows. The full study room schedule has six two-hour booking windows per day (08–10, 10–12, 12–14, 14–16, 16–18, 18–20). For demonstration, each of these real two-hour windows is compressed to a two-minute slot inside a 12-minute repeating cycle. The method reads the real system clock's current minute and second, converts both to a total-seconds-in-the-current-hour value, and takes the modulo of 720 (12 minutes $\times$ 60 seconds) to find the elapsed position within the active cycle. Dividing by 120 (2 minutes $\times$ 60 seconds) then maps that position to one of the six slot indices.

The result is a live, clock-driven slot index that changes every two real minutes and repeats after 12 minutes. It is a genuine stopwatch pattern: measuring elapsed time within a bounded cycle, rather than querying a fixed calendar time. This design allows the study room's six-slot schedule to be fully demonstrated—booking, occupying, and releasing seats.

5.6.4 Complaint Policy

The `ComplaintPolicy` class implements two automatic decisions at complaint submission time using the custom `MyString` infrastructure.

First, it inspects the complaint description against a predefined set of emergency keywords using `MyString.toLowerCase()` and `MyString.containsAny()`. The general emergency set covers: *fire*, *smoke*, *burning*, *sparks*, *electric shock*, *flood*, *burst pipe*, *overflow*, *danger*, and *panic*. Category-specific rules add further triggers: for `ELECTRICITY` complaints, the words *short circuit*, *sparks*, *burning smell*, and *shock* also trigger emergency escalation; for `PLUMBING`, the words *burst*, *flood*, *overflow*, *sewage*, and *blocked main* do the same.

If any match is found, the complaint is assigned `EMERGENCY` priority regardless of category. Students do not need to manually flag urgency; the policy handles escalation entirely based on content.

Second, it maps the category to the appropriate maintenance worker field using the `recommendedWorkerField()` method:

$$\texttt{ELECTRICITY} \rightarrow \texttt{ELECTRICIAN} \quad \texttt{PLUMBING} \rightarrow \texttt{PLUMBER} \quad \texttt{INTERNET} \rightarrow \texttt{INTERNET_TECH}$$
$$\texttt{CLEANING} \rightarrow \texttt{CLEANING}$$

The decision itself is encapsulated in the nested class `DormDecision`, which carries the resolved `PriorityLevel`, a boolean `isEmergency` flag, and any tags appended to the complaint record. This makes the policy layer genuinely decision-making rather than simple record storage.

5.6.5 TimeManager: Clock

The `TimeManager` class in the `utils` package provides the clock infrastructure for all time-sensitive features in Dormatrix. It supports two distinct modes: **normal**, and **Ramadan**.

Real mode. When demo mode is disabled, `nowTime()` simply returns `LocalTime.now()`, making the system directly usable as a live deployment with no code changes.

Ramadan mode. The Ramadan toggle replaces all normal meal windows with Suhoor (03:00–04:30), Iftar (18:00–19:15), and Ramadan Dinner (19:30–21:30). The toggle value is read from `data/foods/config.txt` on startup via `initialize()` and written back on every change via the private `saveConfiguration()` method. This makes the Ramadan state persistent across application restarts — changing the mode does not require re-launching the application.

Meal slot detection. `getCurrentMealSlot()` calls `nowTime()` to obtain the current simulated or real time, then uses the private `isBetween()` helper (which compares `LocalTime` values via `isBefore()` and `isAfter()`) to map the current time to a `MealType` enum value. Weekend breakfasts have an extended window (07:00–10:00 on Saturday and Sunday versus 07:00–09:30 on weekdays). Outside all windows, the method returns `MealType.NONE`, causing any token purchase attempt to be immediately rejected.

5.6.6 LaundryController: Timer-Based Auto-Release

The `LaundryController` implements a stopwatch pattern of a different kind: a real-world `java.util.Timer` to simulate the completion of a wash cycle.

The slot array itself is declared as a `static` field, ensuring that all `LaundryController` instances share a single in-memory representation. The file is loaded only once at first instantiation via the `dataLoaded` flag, preventing redundant file reads while maintaining shared state across roles.

6 Complaint and Scheduling Integration: A Closed-Loop Operational Design

One of the stronger parts of Dormatrix is how the complaint and scheduling parts work together. A complaint is not just stored and forgotten. Once it is assigned, it can move directly into visit planning.

6.1 The Complaint Lifecycle as a Scheduling Driver

When a student submits a complaint, the system saves its category, priority, and suggested worker field. After an auto or manual worker assignment, the complaint becomes ready for scheduling. Before confirming a visit, the scheduler checks whether the student is free in that time slot.

6.2 Auto-Planning: Time-Aware Forward Search

The auto-planning logic looks for a suitable visit slot starting from the current time. It avoids past slots and checks upcoming options first. This keeps the generated schedule practical instead of placing visits at impossible times. Furthermore, auto-planning is constrained to search for free slots between the time 16:00 and 20:00. This is done keeping IUT class routine in mind.

6.3 Bi-Directional Routine Injection

When a visit is confirmed, that visit is also written into the student's routine. So the student can see the complaint visit inside her timetable.

6.4 Scheduling Preference and Reschedule Badges

Students can add simple scheduling notes inside the complaint. Those notes are shown to the attendant as labels, which makes preference handling easier. The student can also apply for rescheduling and give a reasoning behind it.

Complaint-to-Schedule Lifecycle Summary
1. Student submits complaint → the policy logic sets priority and worker field.
2. Worker is auto-assigned or reassigned by worker → the complaint becomes ready for scheduling.
3. System checks worker and student availability → a valid visit slot is selected.
4. Visit is saved → the worker schedule and the student's routine are both updated.
5. Visit is completed → the complaint is resolved and the temporary routine entry is cleared.

7 Utility Infrastructure: The `utils` Package

The `utils` package holds the shared helper classes used across the project. These classes handle terminal rendering, input, time logic, and role mapping.

7.1 ConsoleUtil — ANSI Terminal Primitives

`ConsoleUtil` handles low-level terminal actions such as clearing the screen, moving the cursor, hiding or showing the cursor, and resetting colors. It also has text wrapping support for longer messages.

7.2 TerminalUI and TerminalUIExtras — Bordered Panels and Themed Components

`TerminalUI` builds the visible boxes, headers, and aligned layouts used in dashboards. `TerminalUIExtras` adds extra effects like the matrix-rain animation shown when a role

dashboard opens for the first time.

7.3 BackgroundFiller — Role-Aware Background Rendering

`BackgroundFiller` fills the terminal with the active role's background color so each dashboard feels visually different.

7.4 InputHelper and FastInput — Keyboard and Password Handling

`InputHelper` handles password reading, and `FastInput` keeps the normal input flow simpler and more consistent. Arrow-key support is also tied into the menu system.

7.5 RoleMapper — Role Name Normalization

`RoleMapper` converts role name variations into one standard form so the system can reliably load the right files and dashboards.

7.6 CafeteriaAsciiUI — Animated Progress Bar

`CafeteriaAsciiUI` shows the live progress bar in the cafeteria manager dashboard based on the current meal window.

7.7 TimeManager as a Shared Service

`TimeManager` is kept in `utils` because it is used by more than one module, especially cafeteria, scheduling, and routine-related features.

8 How Dormatrix Behaves Like a Real-World System

Dormatrix was built around situations that actually happen in hall life. Some examples are listed below.

8.1 Time-Gated Cafeteria Purchases

Meal tokens can only be bought during valid meal times. The system also stops duplicate token purchase for the same meal.

8.2 Worker-Student Schedule Conflict Avoidance

Complaint visits are not placed in a student's busy class slot if the routine already shows that time as occupied.

8.3 Laundry Machine Auto-Release

Laundry slots free up automatically after the timer ends, so the system does not depend on manual release.

8.4 Emergency Complaint Escalation

Emergency words in a complaint can raise its priority automatically, which helps attendants react faster.

8.5 Ramadan Schedule Switching

The cafeteria manager can switch the system to Ramadan meal times, and that setting stays saved.

8.6 Store Credit and Due Tracking

If a student's balance is not enough, the remaining amount can be saved as due instead of blocking the purchase completely.

8.7 Fridge Allocation Without

Fridge slots are assigned automatically in order, which keeps the process fair.

8.8 Student Routine as a Living Document

The routine changes when complaint visits are scheduled or moved, so it stays useful instead of static.

9 Module-Wise Implementation Details

9.1 Authentication Module

The authentication module checks the selected role, reads the correct user file, hashes the entered password, and then opens the right dashboard after a successful match.

JLine3 is used here for masked password input. This hides the password from showing in the terminal.

Implementation highlights:

- Users are stored by role in separate files;
- Role names are normalized before file lookup;
- Invalid or cancelled input returns safely to the entry flow;
- The login controller connects the entry screen to all dashboards.

Login rules enforced by the system:

- Passwords are never stored in plain text anywhere. Only the hash is written to file.
- Each role reads from its own separate user file, so a student credential is never checked against a staff file.

- Role names are normalized before the file lookup, so minor variations in the role name string do not break the routing.
- Cancelling or entering blank input during login returns safely to the role-selection menu without throwing an error.

9.2 Dashboard and Terminal UI Module

The terminal UI is one of the main visual strengths of Dormatrix. It uses role-based colors, bordered panels, aligned layouts, and arrow-key or number-based menu movement. The matrix-style animation is also shown once when a dashboard is opened for the first time.

9.3 Room Management Module

This module connects student room information with hall office work. Students can view their current room details and apply for a room change. The student-facing room dashboard shows room number and capacity. If a student has not been assigned a room yet, a message appears pointing her toward the Hall Office. The room-change application form pre-fills the current room and lets the student choose a preferred destination room or floor and provide a brief reason.

On the Hall Office side, the module handles adding new rooms, checking availability, assigning rooms to unassigned students, and reviewing room-change applications.

Room management rules:

- A room can only be assigned to a student if it has available capacity. The occupancy count is checked before any assignment is confirmed.
- When a room-change application is approved, the occupancy count of the old room is decremented and the new room's count is incremented in the same operation.
- If a room-change request is rejected, the student's current assignment stays unchanged. She can submit a new application at any time.
- A student who has not yet been assigned a room is visible to the Hall Office as an unassigned account. The Hall Office assignment screen lists these accounts explicitly.
- Adding a room does not auto-assign anyone. Room assignment is always a deliberate action taken by the Hall Office.
- Hall Office can approve, reject or recommend a room when a room change application is sent.

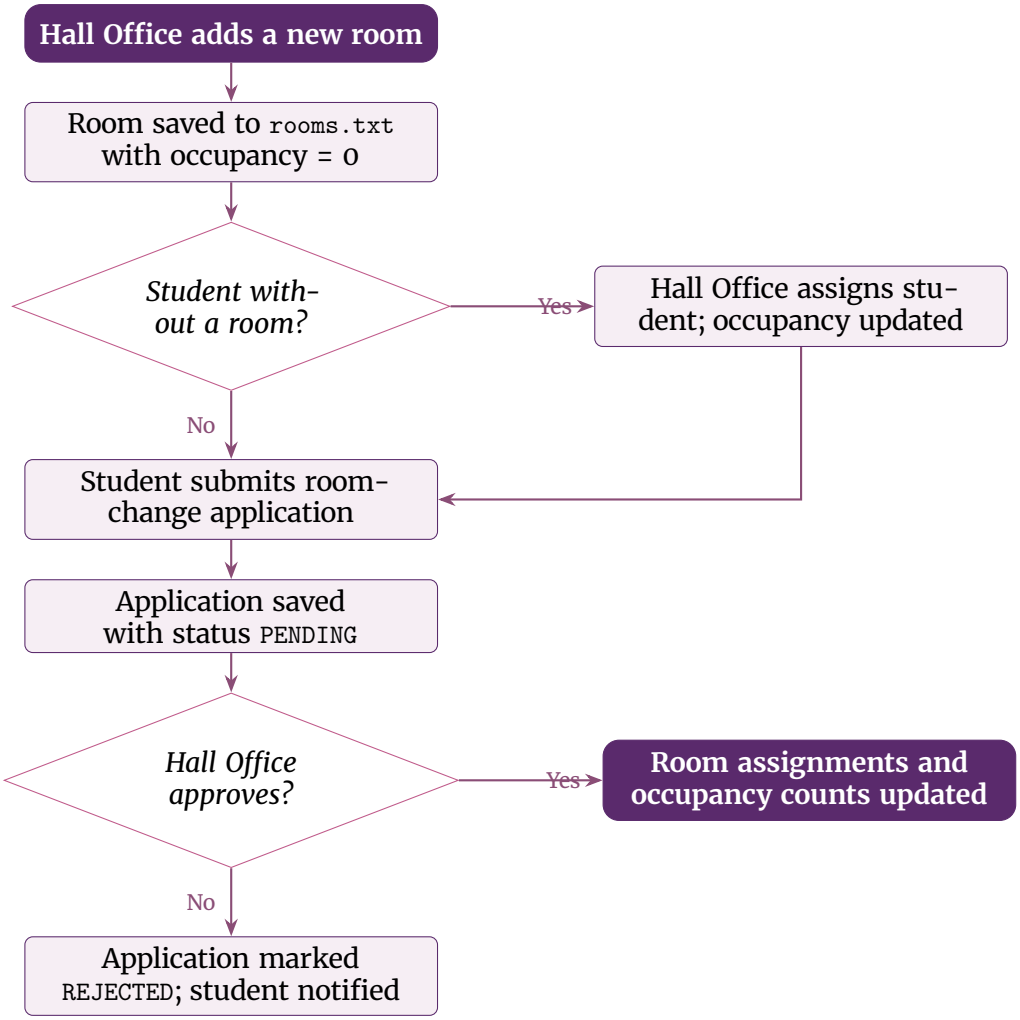

Figure 1: Room management flow: adding rooms, assigning students, and handling change requests

9.4 Facility Booking Module

The facility booking module covers laundry, study room seats, and fridge slots. Laundry uses a timer-based release, the study room uses the cycle clock to decide the active slot, and fridge booking uses first-fit allocation. All three start from the same sub-menu entry point.

Facility booking rules:

- **Laundry:** A student may only hold one active laundry slot at a time. The slot is auto-released after exactly two minutes via a background daemon timer. The release includes a safety check — if the occupant has changed between booking and timer expiry, the timer does nothing. An audit log entry is written on every auto-release.
- **Study room:** Seat availability is always calculated against the current cycle-clock slot index (0–10). Booking is only valid for the duration of the active slot window. When the window ends, all seats are released automatically by the clock.
- **Fridge:** Allocation is fully automatic using the `FirstFitAllocator`. The lowest-

numbered free compartment is assigned. Students cannot choose a specific compartment; this keeps the process fair.

- The slot array is checked before any write, and a `SlotUnavailableException` is thrown if all slots are occupied.

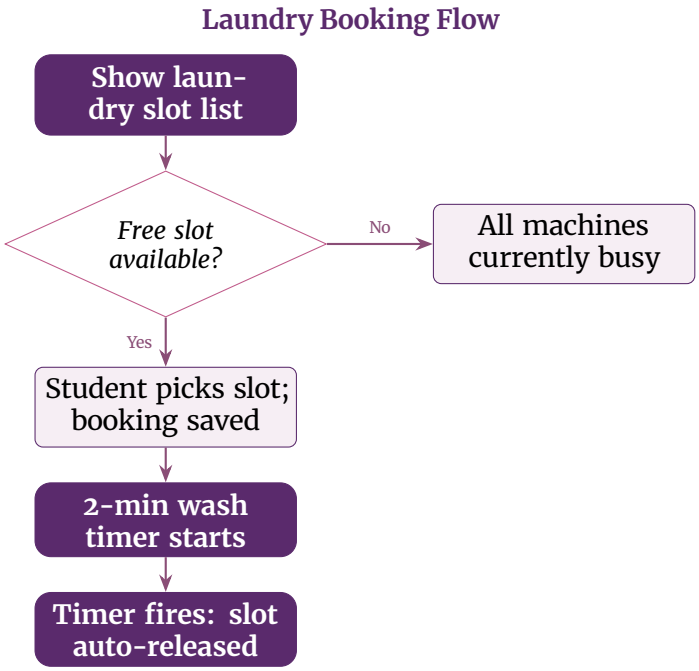

Figure 2: Laundry booking: slot selection, timer start, and auto-release

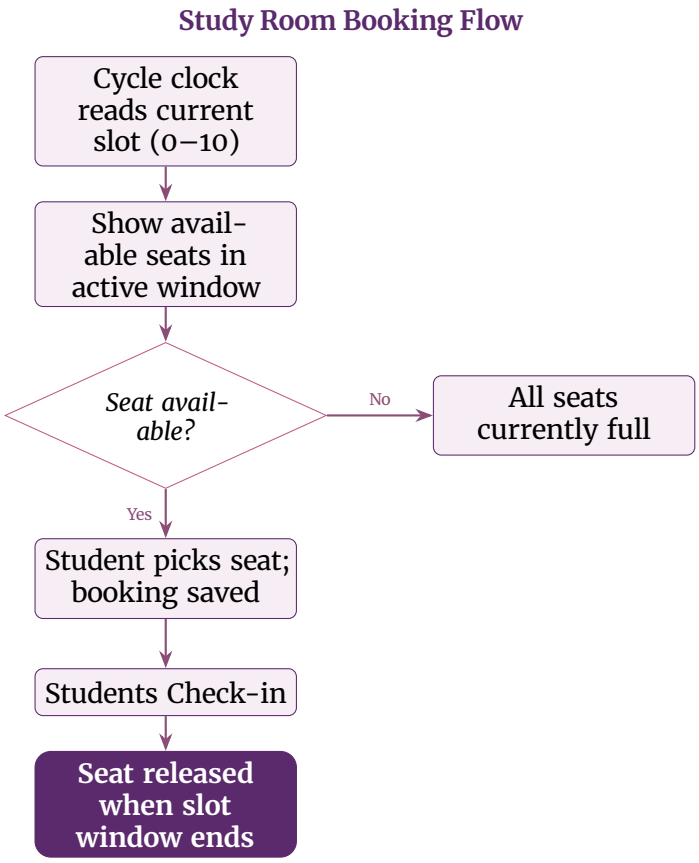

Figure 3: Study room booking: cycle-clock slot detection, seat selection, and auto-release

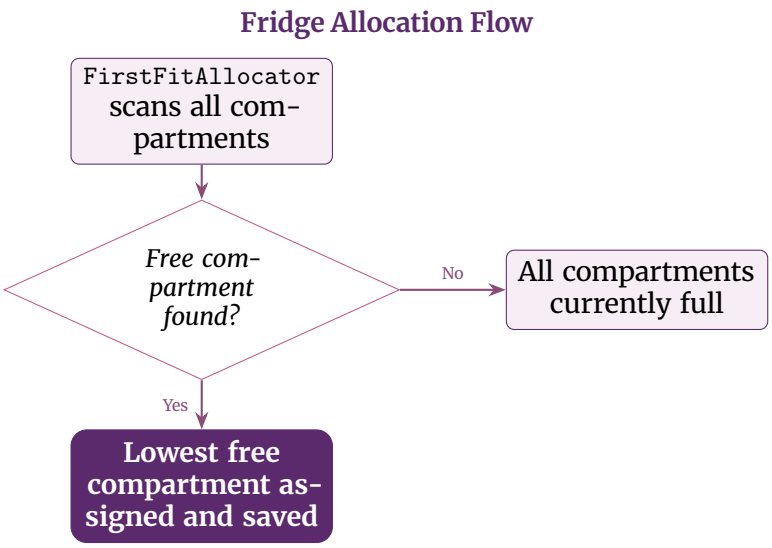

Figure 4: Fridge allocation: first-fit scan and automatic compartment assignment

Study room check-in policy:

To prevent the common *bag-on-seat* problem — where a student reserves a seat but never physically arrives — a two-phase booking model is enforced. Upon reserving a seat, the student has exactly **30 minutes** to return to the Facility menu and select the Check-in option. A background daemon timer starts the moment the booking is confirmed. If the

student checks in within the window, their seat is marked as physically occupied and held for the full slot duration. If no check-in is received before the timer expires, the system automatically releases the seat, logs an `AUTO-RELEASE` audit entry, and the seat immediately becomes available to other students. A student cannot hold more than one seat per active time slot, and the check-in is validated against the current cycle-clock slot index to prevent cross-slot claims.

9.5 Store, Balance, and Due Management Module

The store module handles inventory, cart flow, purchases, balances, dues, and sales history. If a student does not have enough balance, the remaining amount can be saved as due. The purchase goes through regardless so that the student is not blocked outright.

Store and balance rules:

- A student can browse and add items to the cart freely. The balance check only happens at checkout.
- If the cart total exceeds the current balance, the difference is appended to the student's due ledger in `data/store/dues.txt`. The purchase goes through regardless.
- Every completed purchase writes an itemized record to `data/store/sales.txt` and immediately updates the balance file. There is no batching or delay.
- Stock quantities are decremented on purchase. If the quantity of an item hits zero, it is shown as unavailable in the inventory view.
- The Store-in-Charge can view outstanding dues per student and the full sales history at any time from the staff dashboard.

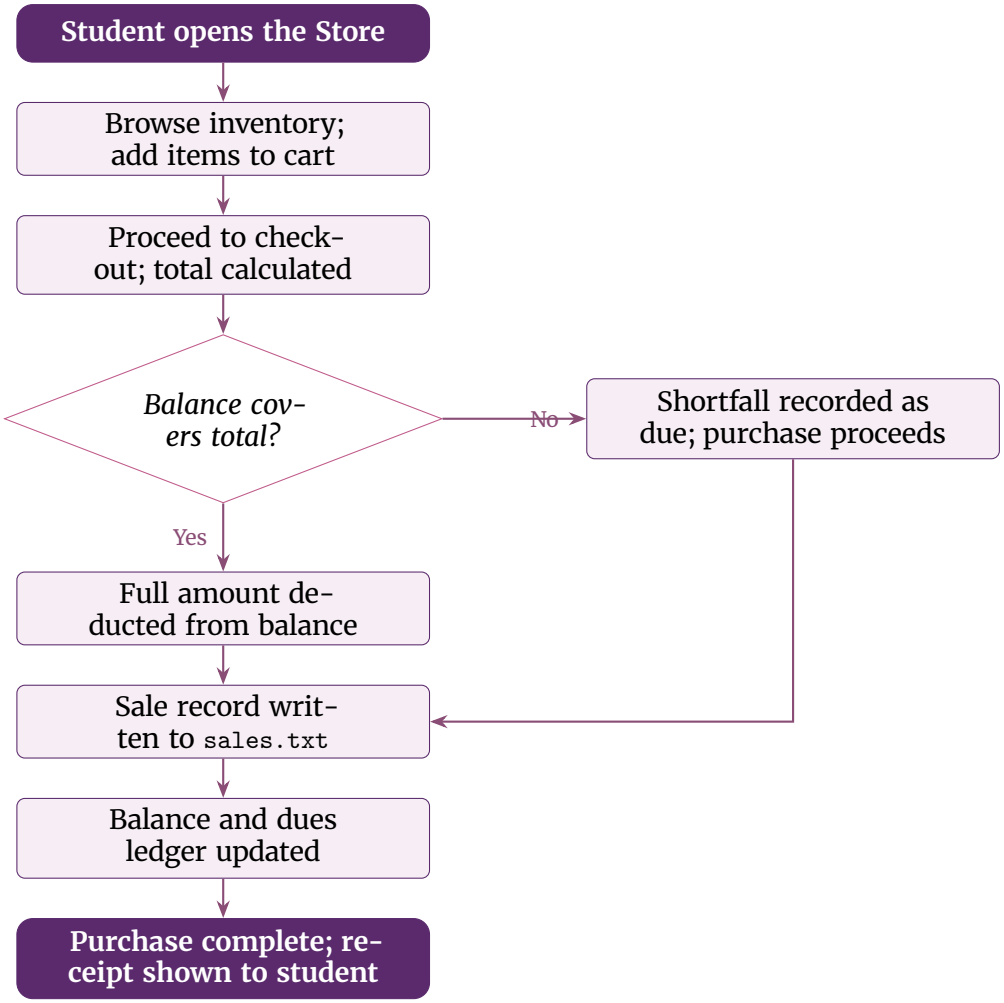

Figure 5: Store purchase flow: cart, balance check, and due recording

9.6 Cafeteria and Meal Token Module

The cafeteria module depends on `TimeManager`. Students can buy tokens only during valid meal times, and duplicate token purchase for the same meal is blocked. The module also handles menu viewing, token verification, and Ramadan mode.

Cafeteria and token rules:

- There are three normal meal windows: Breakfast (07:00–09:30 on weekdays, 07:00–10:00 on weekends), Lunch (12:00–14:00), and Dinner (19:00–21:00). Outside these windows, token purchase is locked.
- In demo mode, the full day from 07:00 to 22:00 is compressed into 20 real minutes, so all three windows open and close within a single session. The scaling ratio is 45 simulated seconds per real second.
- A student can only hold one active token per meal per day. Attempting to buy a second token for the same meal type while one is already active is rejected.
- When Ramadan mode is on, normal meal windows are replaced with Suhoor (03:00–04:30), Iftar (18:00–19:15), and Ramadan Dinner (19:30–21:30). The toggle is persis-

tent across application restarts.

- Tokens have three possible statuses: `Active`, `Used`, and `Expired`. The cafeteria manager verifies a token by entering its ID, which marks it as `Used` if it is still `Active`.
- Token records are written to `data/foods/tokens.txt` with the student ID, meal type, issue timestamp, and current status.

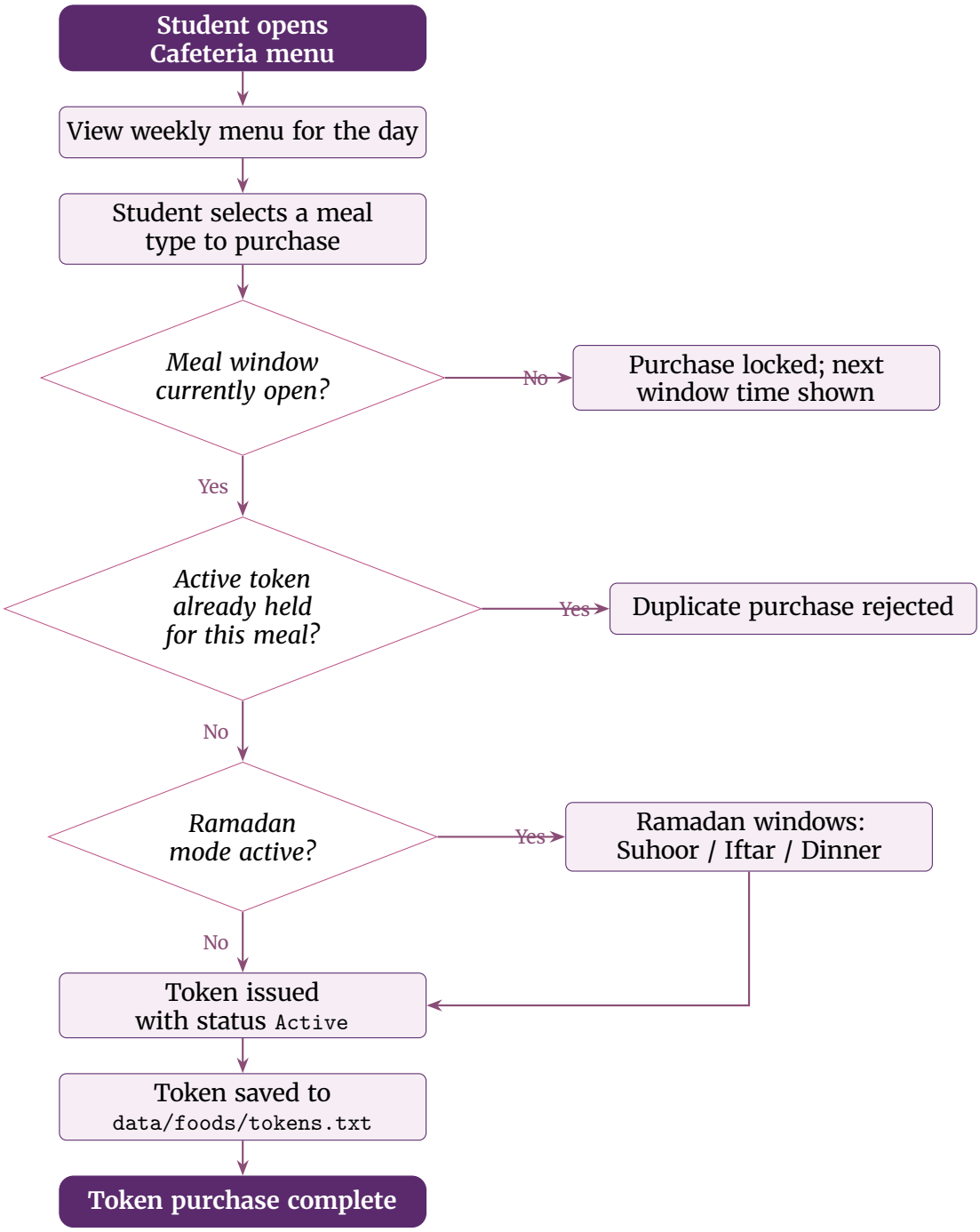

Figure 6: Meal token purchase flow: time window check, duplicate check, and Ramadan mode

9.7 Complaint Management Module

The complaint module follows a clear flow from submission to resolution. The system sets priority and worker type early, stores every status change, and connects the complaint to visit scheduling when needed.

```
SUBMITTED → ASSIGNED → IN_PROGRESS → RESOLVED
```

Complaint rules:

- The `ComplaintPolicy` runs immediately on submission, before the record is saved. The student does not need to manually flag urgency.
- Emergency keywords are checked in a case-insensitive manner using `MyString.toLowerCase()` and `MyString.containsAny()`. General triggers include fire, smoke, burning, sparks, electric shock, flood, burst pipe, overflow, danger, and panic. Category-specific triggers apply on top of those.
- The recommended worker field is derived from the complaint category: Electricity maps to Electrician, Plumbing to Plumber, Internet to Internet Tech, and Cleaning to Cleaning.
- Scheduling checks the student's routine before confirming a visit time. A slot that is already occupied in the routine is skipped automatically.
- Visit records are written into the student's routine file so the student can see the planned visit without needing to check a separate schedule.
- Students can add scheduling preference notes to a complaint. These appear as badge labels when the attendant reviews the complaint, making scheduling easier.
- Students can also request for rescheduling along with a reason which the attendant can check and reschedule.

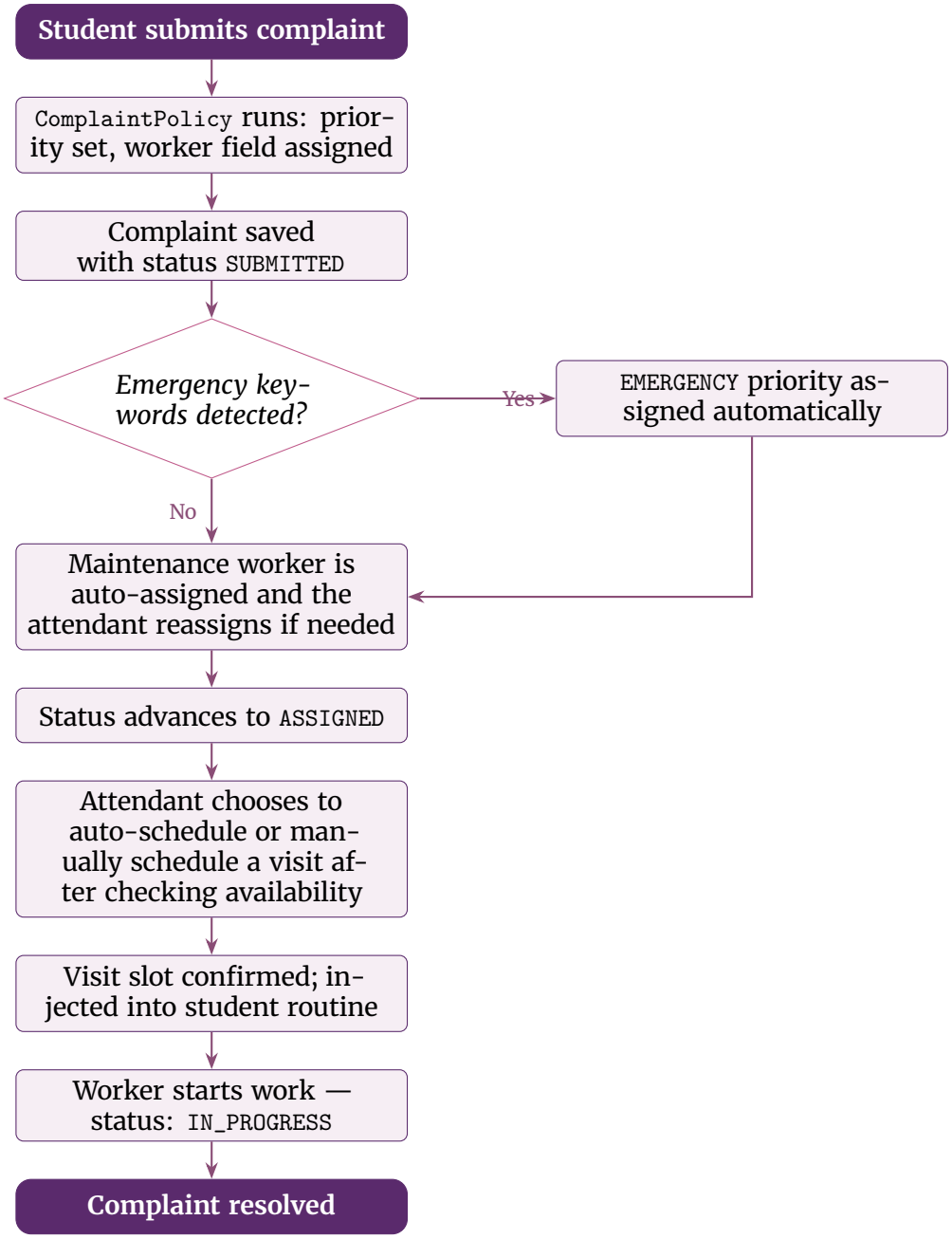

Figure 7: Complaint lifecycle from submission to resolution

9.8 Lost and Found Module

Students report lost items through a timestamped form written to `data/lostItems.txt` with status `UNCLAIMED`. Attendants log found items to `data/foundItems.txt`. Students can claim found items, updating the record to `CLAIMED` with the claiming student's ID and timestamp.

Lost and found rules:

- A student can report any number of lost items. Each report is saved as a separate timestamped record in `data/lostItems.txt` with status `UNCLAIMED`.

- Attendants log found items to `data/foundItems.txt` with a description, timestamp, and initial status `UNCLAIMED`.
- A student can browse all found items and claim one. Claiming writes her student ID and the current timestamp to the found-item record and updates its status to `CLAIMED`.
- Only one student can claim a found item. Once marked `CLAIMED`, the item is no longer shown as available to others.
- Lost-item reports made by a student remain in the system regardless of whether a matching found item exists. The student can view her own lost-item history.
- No automated matching between lost and found items is done. Physical verification is expected before a claim is processed.

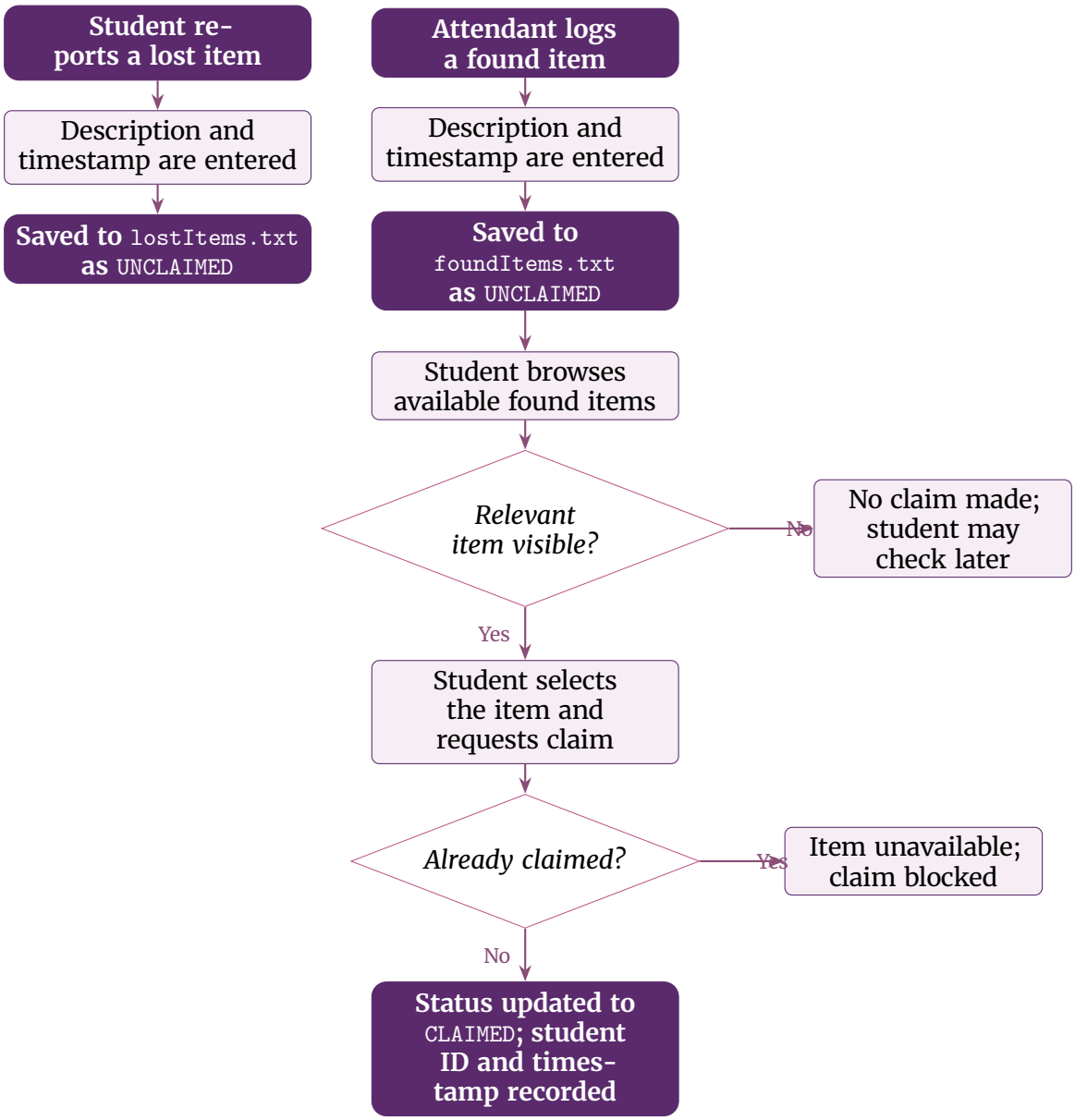

Figure 8: Lost and found flow: lost-item reporting (left) and found-item logging with claim processing (right)

9.9 Announcement Module

Announcements are stored with timestamps and shown in recent-first order. Hall Attendant can post or update an announcement and choose to add an expiry date.

9.10 Routine, Contacts, and Profile Management

The routine module helps students manage their weekly schedule and also supports complaint scheduling. Emergency contacts and profile editing are handled as well, with password verification required before important profile changes.

9.11 Administrative Control Module

The Administrative Control Module provides centralized account management through the Admin dashboard.

Administrative Operations

The module supports user creation, deletion, viewing, and cross-role account searching. This keeps account management separate from the operational parts of the system.

Configuration-Based Access

Administrative credentials are loaded by the `ConfigLoader` class from the `admin.config` file. This makes the system more general and reduces dependence on hardcoded administrative access.

Validation and Storage

Validation is applied before saving account records. Passwords are stored in hashed form rather than plain text.

Architectural Role

The module functions as a control layer for administrative access and system-wide account management.

10 User Flow and Interface

Dormatrix has seven roles and each one gets its own dashboard, its own color, and its own set of actions. However, the overall interaction pattern remains consistent throughout the system: the user selects a role, logs in, reaches the corresponding dashboard, and navigates from there. This section presents the user flow of each role and the general interface experience of the system.

10.1 Entry and Login

When Dormatrix starts, the entry screen displays the title banner and a numbered list of all seven roles. The user may either type the number corresponding to the intended role or use the arrow keys to move through the list and press Enter. This role-selection screen serves as the common starting point for every user before role-specific authentication begins.

Figure 9: Role selection screen

After a role is selected, the login form appears. The user enters the ID and password, where each password character is hidden behind a bullet symbol through JLine3-based masked input. The entered password is processed by the three-channel `HashFunction` and matched against the stored hash in the corresponding user file. A successful match opens the appropriate dashboard after a short animation. An unsuccessful attempt allows the user to retry, and after three consecutive failed attempts the system returns to the role-selection screen.

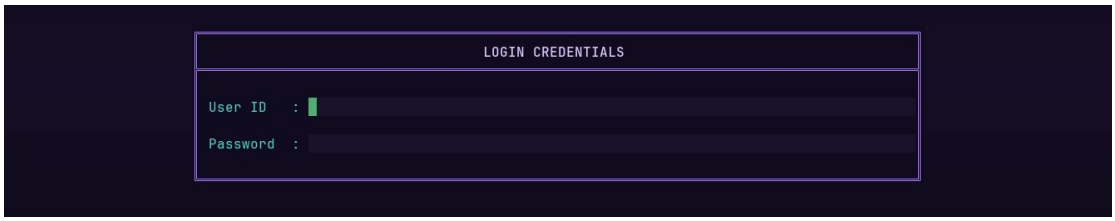

Figure 10: Login form

10.2 Student

The Student dashboard contains eleven options and represents the most feature-rich interface in the system. A student can complete almost all hall-related tasks from this single dashboard.

Figure 11: Student dashboard overview

My Room

This section displays the student's current room information, including room number, floor, section, capacity, and the list of roommates with names and IDs. If no room has been assigned yet, the interface displays a message directing the student to the Hall Office.

Apply for Room Change

This option opens a short application form with the current room already filled in. The student can select a preferred destination room or floor and provide a brief reason. The submitted request is stored with status `PENDING` and appears immediately in the Hall Office review queue.

Book a Facility

This opens the facility sub-menu for laundry, study room, and fridge booking. The student selects the desired facility and follows the relevant booking steps. The booking logic and rules are described in the Facility Booking Module section.

Submit a Complaint

The student selects a complaint category and enters a description. The system processes the complaint through the policy engine before saving it. An optional scheduling note may also be added. The complaint is then forwarded to the Attendant dashboard.

Cafeteria

This section allows the student to view the weekly menu and purchase meal tokens within the current meal window. Time restrictions and duplicate-purchase prevention are handled automatically by the system.

Store and Balance

The student can browse products, add items to the cart, and complete purchases. Balance verification and due recording are managed automatically during checkout.

Announcements

This section displays all hall-wide announcements in reverse chronological order, along with timestamps and expiry dates. Long texts are wrapped automatically according to the terminal width.

Routine

The student's weekly class routine is shown day by day. If a maintenance visit is scheduled through a complaint, the visit appears directly in the routine as a special entry. Regular class entries can be added, edited, or removed, while visit markers remain until the associated complaint is resolved.

Emergency Contacts

Emergency contacts maintained by the Hall Attendant can be viewed directly from the student dashboard.

Lost and Found

The student may report a lost item or claim a found one. Lost-item reporting requires a description, while found items can be browsed from the available list for claim initiation.

Edit Profile

This option allows the student to update name, phone number, and email address. Password changes require the current password to be entered first. All updates are validated before being written back to file.

10.3 Attendant

The Attendant dashboard contains seven options and is primarily centered on complaint handling and hall communication. Complaints are displayed in grouped priority order, ensuring that any complaint marked `EMERGENCY` appears at the top of the queue. The attendant can review a complaint, select an available maintenance worker whose field matches the recommended assignment, and confirm the allocation. The same dashboard also provides access to worker visit scheduling, announcement posting, found-item logging, and emergency contact management.

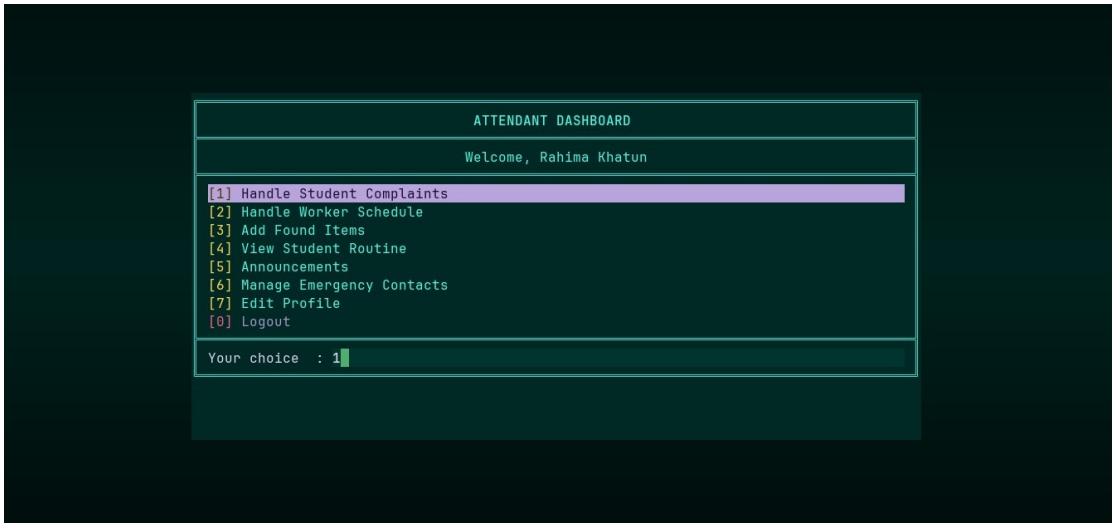

Figure 12: Attendant dashboard and complaint assignment screen

10.4 Maintenance Worker

The Maintenance Worker dashboard is intentionally minimal and contains four options. The worker can view complaints assigned to the relevant field, open a complaint, read its details, and update its state during execution. The status can be advanced to `ASSIGNED` when the task is taken and to `IN_PROGRESS` when the work begins. The status cannot be directly changed to `RESOLVED` from this dashboard. A separate schedule view shows all confirmed visit times for the week.

Figure 13: Maintenance worker task list and visit schedule

10.5 Store-in-Charge

The Store-in-Charge dashboard contains four options focused on inventory control and sales processing. From this dashboard, product lists can be viewed, stock quantities can be updated, student purchase orders can be processed, and both summary and itemized sales records can be reviewed. Every stock modification and completed sale is written to file immediately.

Figure 14: Store-in-charge inventory management and sales summary

10.6 Cafeteria Manager

The Cafeteria Manager dashboard contains five options. It supports weekly menu updates by day and meal type, special event menu posting, and token verification through token ID entry. If a token is still `Active`, verification changes its status to `Used`. The dashboard also contains the Ramadan mode toggle. Once enabled, the normal meal windows are replaced with the Ramadan schedule, and the selected configuration remains preserved even after restarting the application.

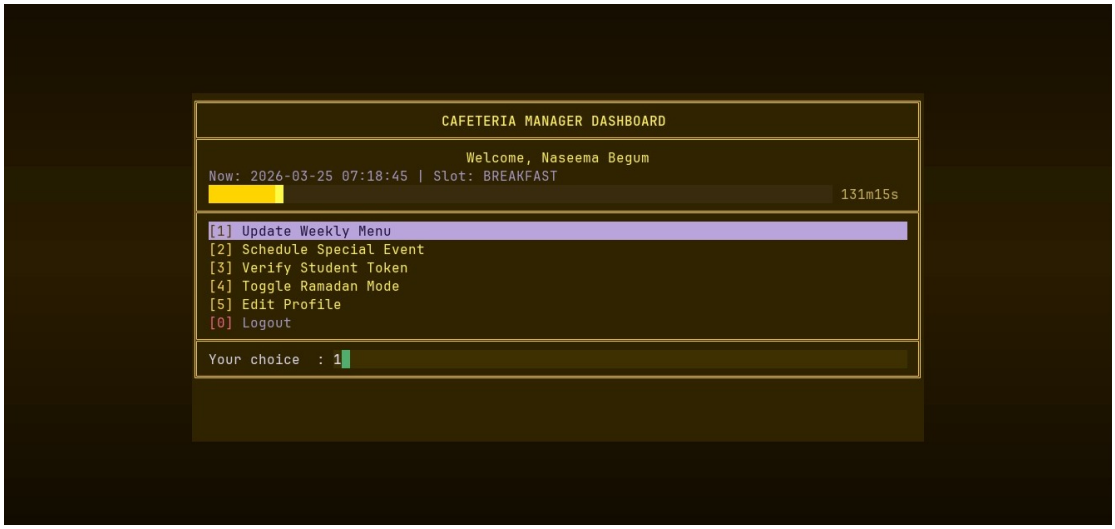

Figure 15: Cafeteria manager menu editor and token verification screen

10.7 Hall Office

The Hall Office dashboard contains six options, all related to room administration. The officer can add new rooms, review room availability together with occupancy counts, assign unassigned students to rooms, and evaluate the queue of room-change applications. Any action that changes a student or room record is followed by a confirmation

step before the data is written.

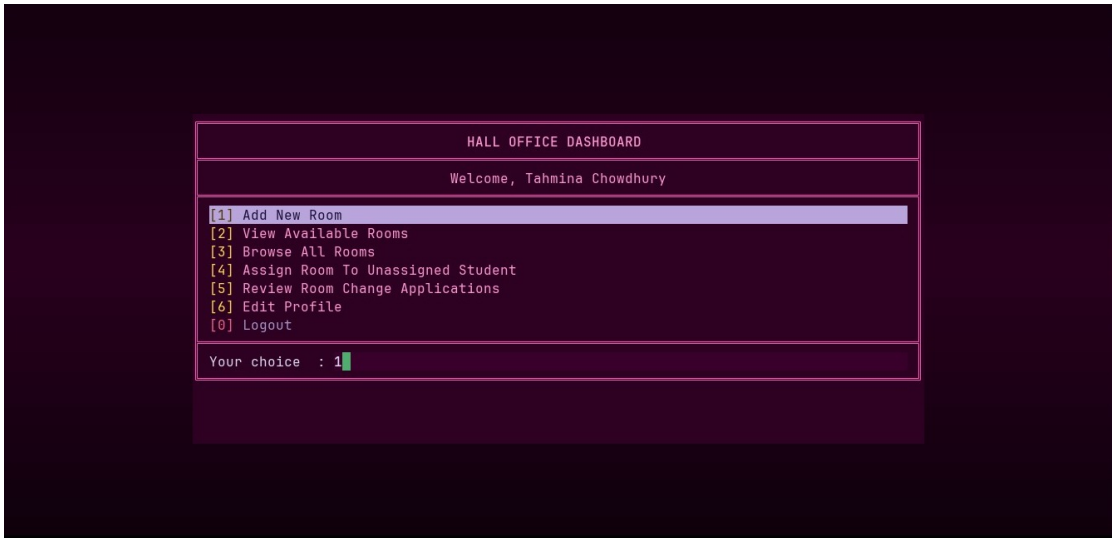

Figure 16: Hall office room management and room-change review screens

10.8 Admin

The Admin dashboard contains four options dedicated to account management. It allows new user accounts to be created for any role, with passwords hashed before storage. It also supports searching accounts by name or ID across all role files, viewing account details, and deleting accounts after confirmation.

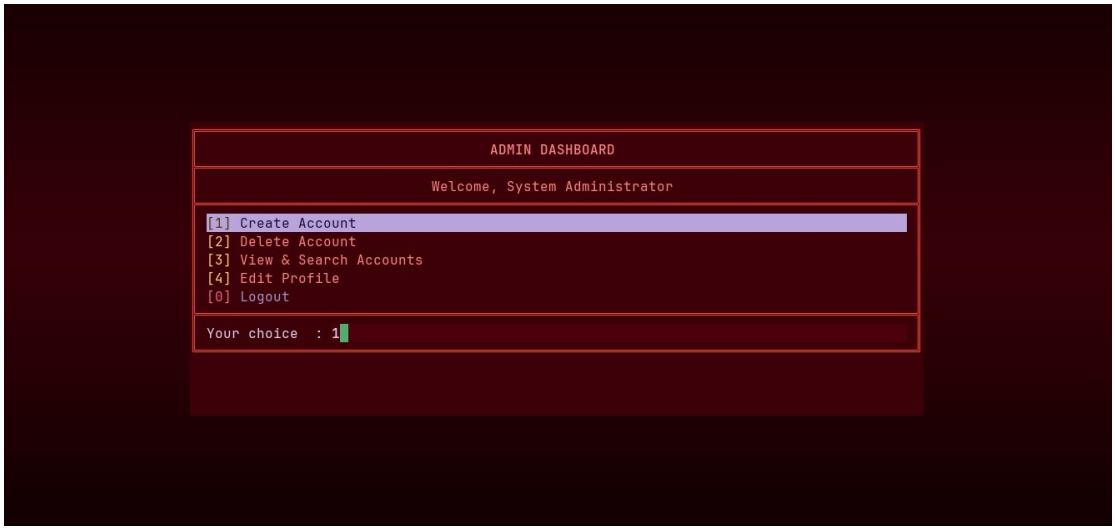

Figure 17: Admin account creation and search screens

10.9 Interface Design

Each role in Dormatrix is presented with a distinct terminal color theme: Navy Blue for Student, Deep Teal for Attendant, Steel Blue for Maintenance Worker, Amber for Store-in-Charge, Hot Pink for Hall Office, Deep Red for Admin, and Golden Yellow for Cafeteria Manager. Panels are rendered using Unicode box-drawing characters and adapt

dynamically to the current terminal width.

Navigation is supported in two ways: direct numeric entry and arrow-key movement with highlighted selection, followed by Enter. Error messages and confirmation prompts appear within the same bordered panel instead of separate pop-ups, ensuring that the interface layout remains stable throughout interaction. Screen transitions clear and redraw the terminal fully. Longer texts, such as announcement bodies and complaint descriptions, are wrapped automatically at either 80 characters or the current terminal width, whichever is smaller.

11 Implementation Evidence and Technical Notes

11.1 Entry Point and Runtime Setup

The `Dormatrix.java` main class calls `TimeManager.initialize()` to load the Ramadan mode flag from `data/foods/config.txt`, then calls `TimeManager.setDemoMode(true)`, which records `System.currentTimeMillis()` as the demo start anchor before launching `MainDashboard`. Starting in demo mode by default ensures that any demonstration begins in a state where all time-sensitive features—laundry timers, meal token windows, study room slots—can be observed immediately without waiting for real clock time.

11.2 Run Environment

The project is distributed with `setup.bat` and `run.bat` batch scripts for Windows. The recommended terminal is WezTerm, which supports ANSI true-color rendering and dynamic size reporting. Windows Terminal with virtual terminal processing enabled is an acceptable alternative.

11.3 Error Handling

The codebase separates normal control flow from exceptional conditions using dedicated exception classes. Domain-specific exceptions such as `SlotUnavailableException`, `InsufficientInventoryException`, `InvalidChoiceException`, and the account validation exceptions are thrown by controllers and caught at the CLI layer, where they are rendered as user-facing error messages rather than stack traces.

11.4 Persistence Transparency

Because all state lives in pipe-delimited text files, the persistence layer is entirely transparent. An evaluator can open any file in `data/`, read and understand its structure without documentation, modify a record manually, and observe the effect immediately on the next application read.

12 Testing and Quality Assurance

12.1 Testing Scope

Testing is an important part of the project. The `tests/` package contains JUnit 5 test files including `UnitTests.java` and `TerminalUITest.java`, which were used to verify core logic and UI-related helper behavior.

12.2 What the Test Suites Cover

The test suites validate both core logic and lower-level utility behavior:

- **Custom collections and string utilities:** `MyArrayList` (add, remove, resize, index bounds), `MyString` (split, containsAny, toLowerCase, trim, intToHex), and `MyOptional` (null safety, isPresent, orElse).
- **Hashing:** `HashFunction` is tested for determinism (same input always produces the same output), distinctness (different inputs produce different outputs with high probability), and length-sensitivity (same characters in different order produce different hashes due to channel 3's position-dependent bit shift).
- **Slot allocation:** `FirstFitAllocator` is tested for correct placement behavior under different occupancy states.
- **Input validation exceptions:** all five account exception classes are tested with both valid and invalid inputs.
- **Role mapping:** `RoleMapper` is tested for all supported role name variant normalizations.
- **Room, routine, schedule, inventory, store, and lost-and-found:** controller logic for assignment, status transitions, entry creation, stock deduction, and claim recording are all validated.
- **Terminal UI helpers:** `TerminalUITest.java` covers padding calculation, gradient rendering, theme application, and box-drawing component output to ensure UI methods produce well-formed ANSI strings.

12.3 Testing Interpretation

The presence of UI helper tests is especially noteworthy because CLI rendering logic is often left entirely unverified. Testing it here shows attention to the full system, not only its domain logic. The test suite also demonstrates that the custom hashing function's security properties were explicitly verified, not merely assumed.

Test execution. The project was tested through JUnit-based automated tests together with manual validation of major user flows.

12.4 Manual Validation

Besides automated tests, we also checked the parts that are easier to judge manually, such as terminal layout, color rendering, arrow-key navigation, and full multi-role flows.

12.5 Known Risk Areas

- File consistency after rapid successive writes, particularly if the application exits mid-write.
- Edge cases in role-based routing when new roles are added or role names change.
- UI rendering correctness at non-standard terminal dimensions (fewer than 80 columns or fewer than 30 rows).
- Long-term maintainability of pipe-delimited file formats as more fields are added to records over time.

13 Contribution

13.1 Team Contribution Overview

Table 9 summarizes the principal contribution areas of each team members.

Table 9: Team contribution matrix

Student ID	Team member	Contribution details
230042114	**Procheta Silvie**	• Development of the store module • Implementation of inventory handling • Development of item browsing, cart, and purchase processing • Implementation of stock update and sales-related operations • Maintenance of the file-based workflow for store records • Implementation of the Store In Charge dashboard functionalities excluding edit profile

Student ID	Team member	Contribution details
230042135	**Khadiza Sultana**	• Development of the authentication module • Development of the cafeteria module • Development of the room management module • Implementation of room application and room-change workflow • Development of profile editing functionalities • Implementation of the Hall Officer dashboard functionalities • Implementation of the Admin dashboard functionalities and administrative control module • Implementation of ConfigLoader • Development of `TerminalUI` • Contribution to most of the shared `utils` package • Development of custom library components including `HashFunction` and `MyString` • Completion of the main portion of unit testing for the core functional areas of the system
230042139	**Sayma Tasnim**	• Development of the slot allocation system • Implementation of the slot allocation algorithms • Development of the lost-and-found module • Contribution to allocation-related logic for booking and assignment • Development of the operational flow for lost-and-found record management • Development of custom library components including firstFitAllocator and slotAllocator

Student ID	Team member	Contribution details
230042141	**Ayman Binta Altaf Nondiny**	• Development of the complaint system • Implementation of complaint workflow and lifecycle handling • Development of complaint policy-related logic and supporting components • Implementation of the complete maintenance worker dashboard • Development of scheduling functionalities • Development of routines • Development of emergency contacts management • Development of the announcement system • Development of `TerminalUIExtras` • Implementation of arrow-selector based UI navigation • Development of the routine UI navigator • Development of custom library components including `MyArrayList` and `MyOptional` • Completion of the later phase of unit testing

13.2 Project Timeline

Figure 18 presents the development timeline of Dormatrix across the fourteen-week project period. The progression reflects the major implementation stages from dashboard foundation and authentication to testing, documentation, and final presentation.

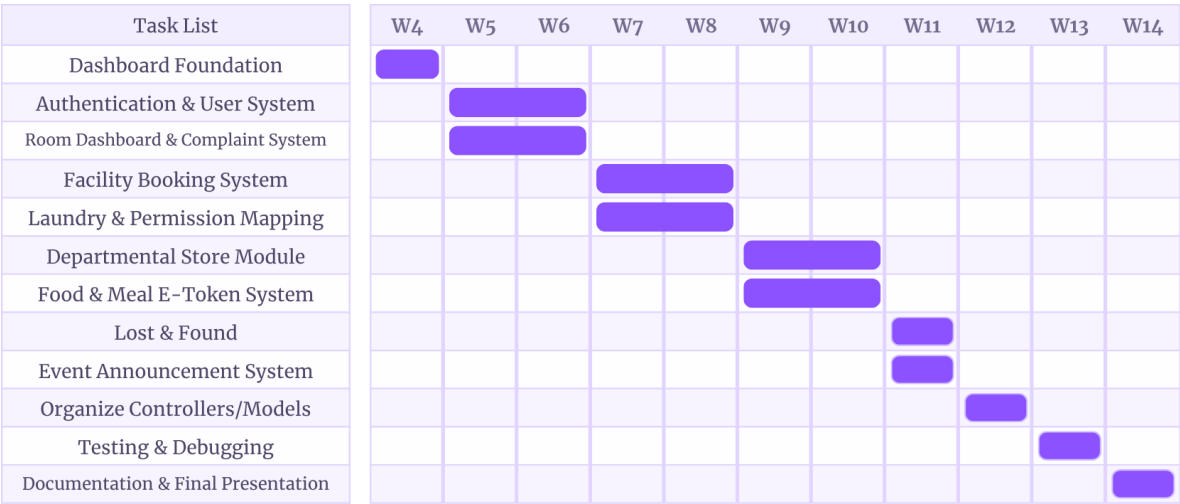

Figure 18: Development timeline of Dormatrix across the project duration

14 Future Work

14.1 Current Limitations

- The system is single-user at runtime. Concurrent access to the same text files by multiple application instances is not handled and could result in lost writes.
- The Windows batch scripts limit the project to Windows environments without modification.
- Text-file storage becomes harder to manage as data volume grows. Long files increase read and write times with no indexing.
- Some UI features require a terminal that properly supports ANSI 24-bit color and cursor-movement sequences.
- The custom hash function lacks per-user salt values, making it vulnerable to precomputed rainbow-table attacks at scale.

14.2 Future Work

- Migrating from flat files to SQLite, which would provide indexed lookups, transactional writes, and safe concurrent access without requiring a separate server.
- Introducing file locking for the multi-user concurrent access case.
- Adding a lightweight web or GUI frontend while retaining the CLI as an administrator-facing mode.
- Integrating in-application notifications for complaint status updates, expiring meal tokens, and new announcements.
- Extending analytics to show complaint trends by category, store demand forecasting, or room occupancy history over time.
- Expanding the slot-cycle clock in `SlotAllocator` to support a configurable mapping from the 12-minute cycle to real two-hour windows, making the transition from demo mode to real deployment a single constant change.
- Improving the store module by grouping items into structured categories such as groceries, stationery, and daily essentials to enhance usability and inventory management.
- Extending the administrative system to support dynamic role creation, permission assignment, and customizable dashboard components for different user roles.
- Refining the fridge allocation system by introducing a compartment-based model, allowing users to be assigned specific sections instead of general slots.
- Enhancing the laundry module by incorporating practical constraints such as usage duration tracking, queue handling, and priority-based allocation.
- Adding a rating and feedback system for maintenance workers to improve accountability and service quality.

- Linking the cafeteria management system with the hall office to support food inventory tracking, including items such as meat, vegetables, and other supplies.
- Introducing a cafeteria food review system to collect student feedback and improve overall meal quality.

15 Personal Reflections

15.1 Procheta Silvie (230042114)

I was responsible for designing and implementing the departmental store module of Dormatrix, which involved inventory management, transaction handling, and tracking student balances and dues. This component required careful structuring of file-based data operations to ensure consistency and reliability without database support. Through this process, I developed a deeper understanding of modular system design, data abstraction, and real-world transaction workflows. I also enhanced my problem-solving skills by addressing challenges related to data integrity and system integration. Overall, this experience significantly strengthened both my technical competence and my ability to contribute effectively within a collaborative software development environment.

15.2 Khadiza Sultana (230042135)

I was responsible for a significant portion of this project, beginning with the authentication system. From my experience with previous projects, I realized that many important values were often hardcoded, which was not secure. This led me to the idea of storing such information in a local configuration file that would not be uploaded to GitHub, while a ConfigLoader would connect it to the system. This became the basis of the admin creation mechanism. At IUT, we were also given individual emails and passwords when we first arrived, which influenced my decision to make account creation admin-controlled. Designing the HashFunction was challenging because I was not very familiar with hashing at the time, so I tried to keep it simple. Later, I used JLine3 to mask passwords and added regex restrictions for fields such as email and phone number during account creation. Since not all roles have email addresses, role-wise mapping was also ensured in authentication and admin-based account creation.

I was also responsible for the Cafeteria Module, where I introduced an additional role to make the system more realistic. The goal was to make the module dynamic and aligned with IUT meal schedules, such as longer breakfast hours on weekends and different timings during Ramadan. For this, I implemented a clock-based TimeManager file where these settings remained persistent. Meal token validation was designed to be handled by a separate person or device rather than through student accounts, which led to the creation of the CafeteriaManager role. The same role was later used for menu management as well, since I felt that every system flow should have a responsible role through which information is added and maintained.

I also developed the room management module. It was kept simple. In IUT, the Hall Officer normally assigns our rooms, and the process of applying for a room change can

be tiring. Last semester, when we went to submit a room change application multiple times, we were repeatedly told that it was not the right time. That experience led me to think of implementing this feature in the system.

Towards the later stage of the project, I began working on the terminal based UI elements. To be honest, this was the most difficult part of my contribution and the part where I spent the most time. Initially, my idea was to make the dashboards visually appealing and give each role a designated color theme. However, I did not realize how difficult terminal-based UI implementation would be. Manipulating the terminal was challenging, but since it was not impossible, I kept working on it and eventually implemented it. Later, when our supervisor suggested more UI-related ideas, I became even more motivated to expand this part further. This was where the implementation of TerminalUI came into place. I used ANSI escape sequences and different methods to build its functionality. These methods were later used in the dashboards to create terminal-based UI features such as animations, transitions, art and effects, live previews, and arrow selectors. Later, Nondiny also added many things to the terminal UI from her side. In many ways, we inspired each other. I also later added an animated clock to the cafeteria module.

Lastly, I added unit test cases for all the modules, which was later continued by Nondiny as well. We learned how to write basic unit test cases this semester, and since we had learned it, I thought of applying it here as well to better follow the software lifecycle process.

In general, my goal for the project was to make sure that we built something that was not overly complex, yet still meaningful. I wanted to ensure that we developed it diligently and that, regardless of how much we accomplished, we put genuine effort into it. I also wanted to become more familiar with the environment of software development.

15.3 Sayma Tasnim (230042139)

I was assigned to design the Facility Booking Module and the Lost and Found system of our project Dormatrix.

The facility module manages three shared dormitory appliances — study room seats, fridge slots, and laundry machines — each with distinct allocation logic. For the study room, I implemented a cycle-clock-based time slot system with a two-phase check-in mechanism to address the real-world bag-on-seat problem. The laundry system uses a timer-based auto-release after a fixed wash cycle, while fridge slot allocation is handled using the First-Fit algorithm. All three components share a timestamp-based persistence model, ensuring a fair booking system for every dorm-residents. For the Lost and Found system, I implemented a relational reporting flow where lost items and found items are stored separately and linked by ID, along with a claim verification mechanism that updates item status upon a valid claim request. There is role-based access permission here making the module consistent with real-world scenarios. Through this process, I developed a deeper understanding of concurrent resource management, timer-based automation, and the challenges of handling conflicts across a shared structure. Debugging issues such as error in file-reading, stale slot data persisting across restarts and array index mismatches in claim verification significantly strengthened my ability to

trace and resolve logic-level bugs in a multi-module system. I also gained hands-on experience with version control using Git and GitHub — managing and coordinating code integration across team members — which taught me the importance of consistent commit practices and clear communication when working collaboratively on a group codebase. The combination of technical depth and collaborative effort in this project improved both my ideation abilities and development skills. Also, being a resident of the dormitory myself, I could directly relate to the problems our features aim to solve. That personal connection made this project feel meaningful beyond just course work.

15.4 Ayman Binta Altaf Nondiny (230042141)

I initially started with complaint module. I gave a lot of thought into it before I started coding. It made me more aware of real life scenarios and hence motivated me to make it more user intuitive. Practicality was my goal.

What was initially just complaint submission, viewing and proper assignment, turned more real-life with the addition of scheduling. This part became more close to me.

I added a 24-hour weekly routine thinking that student would be more intrigued to use it. While in one way it aids in making Student life more disciplined, it also aids in complaint scheduling easier for Hall Attendant. To me, it became a perfect example of a unified design approach, where a single feature addresses the needs of multiple users. I made sure that the attendant can not schedule a routine beyond 8 am to 8 pm. Also, the attendant can not see what is written in the routine slots- to attendant, these are just 'BUSY' slots. Next what came to my mind is 'What is the student does not use the routine and write into it?'. This made me include the student's contact below the routine, so that the attendant can contact the student if the student's routine is too empty or full. This also made me constraint the auto-scheduling from 4 pm to 8 pm as we are almost always busy from 8 am to 4 pm due to classes.

For maintenance workers, I also added a routine to show their duty days and if a complaint scheduling is done, I added it to the maintenance worker routine.

I have also done so much more. One which I will particularly remember is adding navigation to student routine. It made the whole thing so user friendly. I was afraid that I might not be able to implement it, I almost accepted it will not be feasible, but, somehow, I did it! I did not want Khadiza to do all the navigation work, so I partly started implementing it, and I am happy for it.

Starting from Khadiza, I learned a lot from her. She motivated me to start doing my part of the project early. She started quite early, and I felt guilty seeing her working alone. Hence I started doing alongside her, and I came so far. She motivated me to never be satisfied with your project- that there are always chances for enhancement. They she implemented terminals and terminal UI, I got awed. Initially she had just Authentication and Room Management. Later on, she took hold of Cafeteria Management. But the way she implemented them, and what she made out of it, motivated me greatly to enhance my parts further and further.

I also took hold of announcements somewhere in the middle. Initially, it was just posting

announcements and viewing it. But then i made it more practical to hall life using expiry date and an update option. Announcement is a real problem in this hall. At times, I don't even hear what they are saying while at times, there are no announcements at all. Although it is not like complaint system is any better either. With everything being done manually, and no proper record system, thing get real slow and messy.

I, furthermore, added emergency contacts which, while adding, made me think why aren't we given any emergency contact information? My friend once broke her leg, and she literally had no emergency contacts to contact in IUT. Hall security ? - no contact. Hall Attendant? - no contact. Hence I added this section for real convenience.

I leaned so much during the whole duration of this project - how to tackle large projects, how to think, how to navigate, importance of starting early, importance of technical knowledge, importance of always aiming for betterment, and so much more. I became more comfortable with handling Github. I learned to accept my lacking, but never giving up. I learned that I should accept challenges and discomfort as a part of improvement.

Finally, I don't plan on giving it up yet. I plan to enhance it more, and I plan to do it with Khadiza. Hopefully, one day, we will be able to turn it into an application which will be usable in any dormitory.

16 Conclusion

Dormatrix brings many everyday hall tasks into one system and organizes them through role-based dashboards. It covers room work, complaints, facilities, store operations, cafeteria services, announcements, routines, and account management in a way that feels connected.

The project also reflects our own implementation effort. We used file-based storage, kept external libraries minimal, and built several helper classes ourselves. Features like complaint scheduling, meal-time logic, and timer-based booking release also made the system more practical and easier to demonstrate.

Most importantly, the project stayed close to the real problems that inspired it. That is what makes Dormatrix feel like our project and not just a generic management system.